

# Introdução à Lógica & Algoritmos

O ponto de partida da programação: pensamento estruturado, decomposição e resolução de problemas.

```
console.log("Olá, mundo! Iniciando a jornada dev.");
```

## Quem é o Professor?

- **Jeronimo Silva** (Contato: 85 9 9129–2744)
- Trajetória multidisciplinar: Física, Engenharia, Idiomas e Tecnologia da Informação.
- Graduando em Análise e Desenvolvimento de Sistemas (Faculdade Senac).
- Foco: Autonomia prática, raciocínio lógico e preparação profissional contínua.

# Por que JavaScript + Ecosystem Desktop?

- **Tecnologia das Gigantes:** A base de softwares de classe mundial como VS Code, Discord, Spotify e WhatsApp.
- **Linguagem Mais Utilizada:** Líder global de adoção há mais de uma década (*Stack Overflow Survey*).
- **Versatilidade Real:** A mesma linguagem opera no Navegador (Front-end), no Servidor (Node.js) e no Desktop (Electron).
- **Multiplataforma:** Um único código roda de forma nativa no Windows, Linux e macOS.

## INTEGRAÇÃO

# Quem são vocês?

Vamos quebrar o gelo e alinhar nossos objetivos:

- Qual o seu nome e sua área de interesse?
- Já teve algum contato anterior com código ou tecnologia?
- Qual o seu principal objetivo ao concluir esta formação?

# Nosso Acordo de Laboratório

- **Compromisso e Frequência:** O aprendizado em programação é cumulativo e prático. Cada aula constrói a base da seguinte.
- **Espírito de Pesquisa:** Erros de sintaxe são normais. Aprender a ler mensagens de erro faz parte do ofício.
- **Ambiente Colaborativo:** Tirar dúvidas em sala e apoiar os colegas fortalece o aprendizado coletivo.

VISÃO GERAL · 200 HORAS

# Trilha de Formação Técnica

## UC1 • Sistemas de Informação (72h)

Lógica algorítmica, variáveis, controle de fluxo e CLI com Node.js

## UC2 • Banco de Dados (72h)

Modelagem relacional (MER/DER), linguagem SQL e persistência

## UC3 • Testes e Manutenção (36h)

Qualidade de código, depuração, boas práticas e Git/GitHub

## UC4 • Projeto Integrador (20h)

Aplicação Desktop completa ponta a ponta (Electron + Banco)

# UC1: Desenvolvimento de Sistemas

*O cérebro lógico e algorítmico do desenvolvedor:*

- **Fundamentos:** Algoritmos, variáveis, tipos de dados e controle de fluxo ( `if/else` , loops).
- **Modularização:** Funções puras e rotinas reutilizáveis.
- **Ferramental:** Node.js + JavaScript moderno + `@inquirer/prompts`.
- **Entrega Prática:** Criação de sistemas interativos em linha de comando (CLI).

## UC2: Banco de Dados

*Onde as informações ganham persistência, estrutura e segurança:*

- **Modelagem:** Entidades, atributos e relacionamentos (MER e DER).
- **Linguagem SQL:** Comandos fundamentais ( `SELECT` , `INSERT` , `UPDATE` , `DELETE` , `JOIN` ).
- **Integridade de Dados:** Chaves primárias, estrangeiras e normalização.
- **Entrega Prática:** Banco de dados relacional integrado à aplicação.



## UC3: Testes & Manutenção

*A garantia de estabilidade, previsibilidade e colaboração:*

- **Prevenção de Falhas:** Encontrar e corrigir comportamentos inesperados.
- **Código Limpo:** Refatoração, legibilidade e manutenibilidade.
- **Controle de Versão:** Fluxos profissionais com **Git e GitHub**.
- **Entrega Prática:** Software testado, documentado e versionado profissionalmente.

## UC4: Projeto Integrador

*A consolidação prática de toda a formação:*

- **Problema Real:** Solução desenhada para resolver um gargalo real do mercado.
- **Stack Completa:** Interface Desktop (**Electron**) + Regras de Negócio (**Node.js**) + Dados (**SQL**).
- **Portfólio:** Código público no GitHub e aplicação funcional para demonstração.

# O Profissional de Sistemas

- **Mais que digitadores de código:** Nossa missão central é resolver problemas práticos de forma automatizada e lógica.
- **Atitude Autônoma:** Capacidade de buscar documentações, testar hipóteses e depurar problemas.
- As linguagens e ferramentas evoluem com o tempo; sua capacidade de **raciocinar logicamente** permanece.

# Tour Técnico: Identificando Gargalos

Vamos circular pelas instalações da unidade Senac Centro (Biblioteca, Cantina, Recepção):

## 1. Observe a Rotina

Como as pessoas interagem com os serviços? Onde elas aguardam atendimento?

## 2. Mapeie Gargalos

Onde há filas demoradas, atrasos ou retrabalho manual com fichas e papéis?

## 3. Pense como Dev

Que tipo de software, tela ou automação resolveria essa dor de forma definitiva?

**Objetivo:** Mapear problemas reais do mundo real que inspirarão nossos projetos durante o curso!

# O que é um Algoritmo?

É uma **sequência finita, ordenada e não ambígua** de passos para resolver um problema ou executar uma tarefa.

- **Início e Fim:** Um algoritmo sempre possui término previsível.
- **Não Ambíguo:** Cada instrução tem sentido único e claro.
- **Ordenado:** A sequência exata das etapas altera o resultado final.

O computador executa exatamente o que você programa, nunca o que você supõe.

O Tripé da Computação



**Entrada:** Dados fornecidos pelo usuário ou sensores.



**Processamento:** Regras lógicas, decisões e cálculos.



**Saída:** Informação apresentada na tela ou gravada no banco.

# Algoritmos Fora do Computador

```
# Algoritmo: Trocar Lâmpada Queimada  
1. Posicionar escada embaixo do bocal  
2. Desligar o interruptor de energia  
3. Subir com segurança na escada  
4. Desenroscar a lâmpada danificada  
5. Enroscar a nova lâmpada até fixar  
6. Descer, ligar o interruptor e testar
```

**Sequência Lógica:** Se você ligar a energia antes de enroscar, haverá choque elétrico.

**Precisão:** Cada instrução depende rigorosamente do sucesso da etapa anterior.

# Os 4 Pilares do Pensamento Computacional

## 1. Decomposição

Quebrar um problema complexo em partes menores e gerenciáveis.

## 2. Padrões

Identificar repetições ou semelhanças com problemas já conhecidos.

## 3. Abstração

Focar no que é relevante e ignorar os detalhes secundários.

## 4. Algoritmo

Construir o passo a passo exato e ordenado para resolver a tarefa.

Essa é a linguagem universal de todo profissional de tecnologia.

# O Algoritmo do Robô Humano

1. Em duplas ou trios, escolham uma tarefa cotidiana simples no laboratório (ex: *pegar a caneta no estojo e escrever o nome no quadro*).
2. Escrevam no caderno o passo a passo literal para executar a tarefa.
3. Um colega de outro grupo atuará como o robô e seguirá estritamente as instruções escritas.



## Próximo Passo: Setup na Nuvem & GitHub

Hoje deixaremos o nosso ambiente profissional 100% configurado para a jornada:

### 1. Criar Conta no GitHub

O passaporte e portfólio universal do programador. Todos os projetos que desenvolveremos viverão aqui.

### 2. Ativar o GitHub Codespaces

VS Code profissional executando direto na nuvem. Sem tempo perdido com instalações locais!

 **Amanhã à tarde:** Nossas primeiras linhas de código JavaScript e Node.js no terminal!